

Theory of Computation

Regular Languages and Regular Expressions

Samit Biswas¹

¹Department of Computer Science and Technology,
Indian Institute of Engineering Science and Technology, Shibpur
Email: samit@cs.iiests.ac.in

Table of Contents

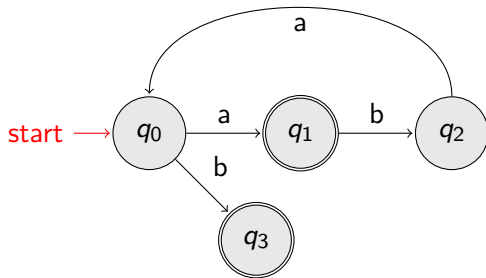
- 1 Single Final State for NFAs and DFAs
- 2 Some Properties of Regular Languages
- 3 Regular Expressions

Single Final State for NFAs and DFAs

- Any Finite Automaton (NFA or DFA) can be converted to an equivalent NFA with a single final state.

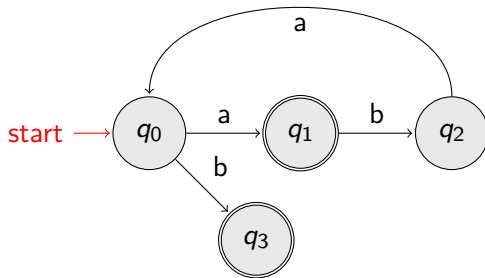
Single Final State for NFAs and DFAs (Example:)

NFA

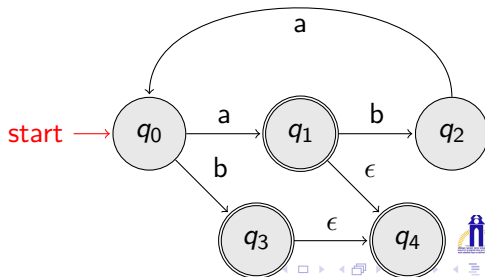


Single Final State for NFAs and DFAs (Example:)

NFA



Equivalent NFA



Single Final State for NFAs and DFAs (In general)

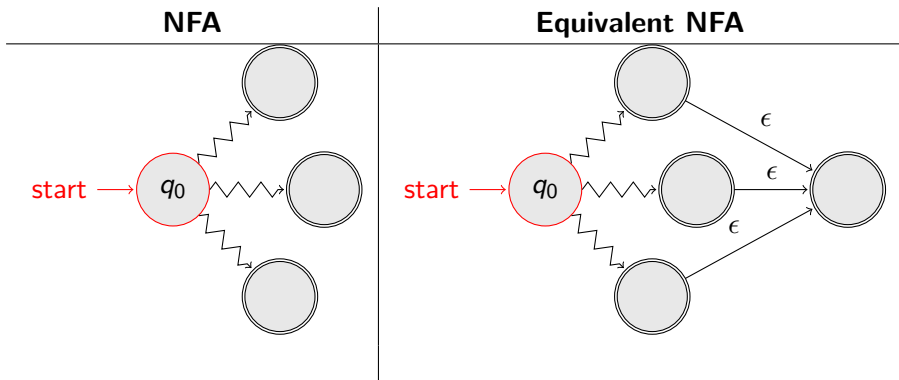


Table of Contents

- 1 Single Final State for NFAs and DFAs
- 2 Some Properties of Regular Languages
- 3 Regular Expressions

Some Properties of Regular Languages

- For regular languages L_1 and L_2 we will prove that:

$$\left. \begin{array}{l} \text{Union: } L_1 \cup L_2 \\ \text{Concatenation: } L_1 L_2 \\ \text{Star: } L_1^* \end{array} \right\} = \{\text{Are Regular Languages}\}$$

Some Properties of Regular Languages

- For regular languages L_1 and L_2 we will prove that:

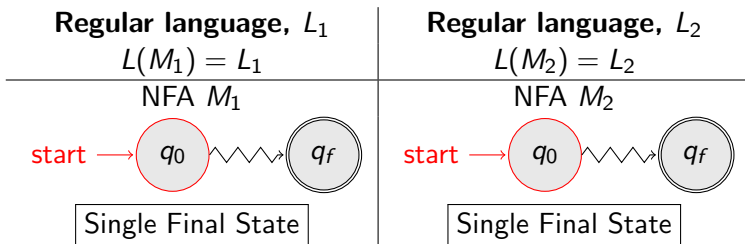
$$\left. \begin{array}{l} \text{Union: } L_1 \cup L_2 \\ \text{Concatenation: } L_1 L_2 \\ \text{Star: } L_1^* \end{array} \right\} = \{\text{Are Regular Languages}\}$$

- We say:
Regular languages are **closed** under

$$\left. \begin{array}{l} \text{Union: } L_1 \cup L_2 \\ \text{Concatenation: } L_1 L_2 \\ \text{Star: } L_1^* \end{array} \right\}$$

Some Properties of Regular Languages

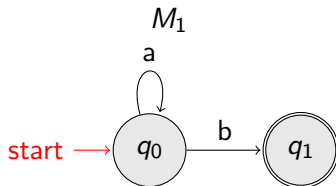
- Properties:



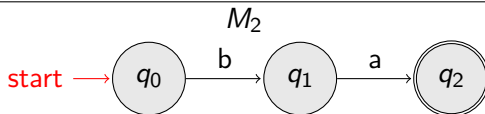
Some Properties of Regular Languages

- **Example:**

$$L_1 = \{a^n b\}$$



$$L_2 = \{ba\}$$

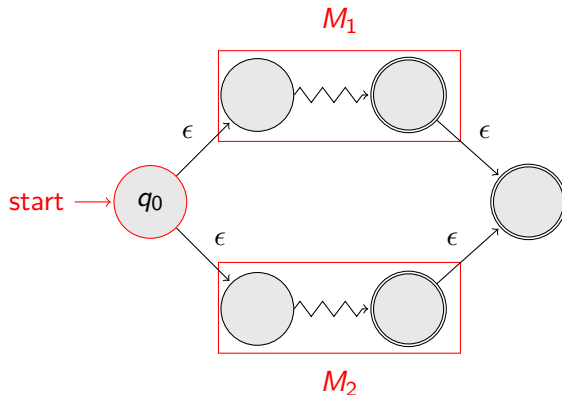


Some Properties of Regular Languages

- **NFA for $L_1 \cup L_2$**

Some Properties of Regular Languages

- NFA for $L_1 \cup L_2$

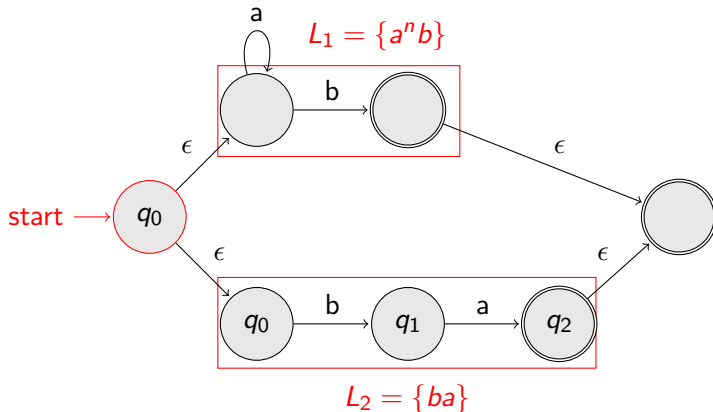


Some Properties of Regular Languages

- **Example:** NFA for $L_1 \cup L_2 = \{a^n b\} \cup \{ba\}$

Some Properties of Regular Languages

- **Example:** NFA for $L_1 \cup L_2 = \{a^n b\} \cup \{ba\}$



Some Properties of Regular Languages

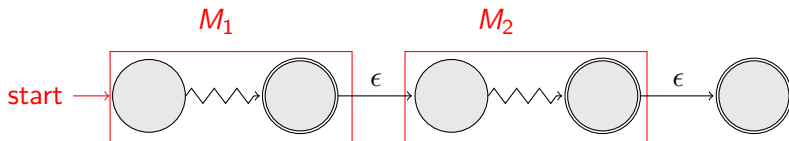
- **Concatenation:**

NFA for L_1L_2

Some Properties of Regular Languages

- Concatenation:**

NFA for L_1L_2

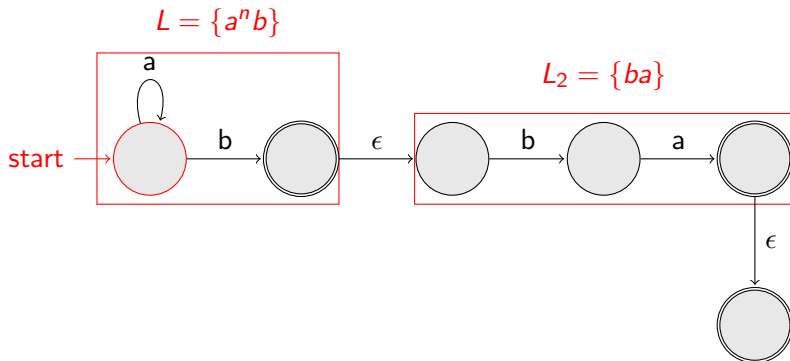


Some Properties of Regular Languages

- **Example:** NFA for $L_1L_2 = \{a^n b\}\{ba\} = \{a^n bba\}$

Some Properties of Regular Languages

- **Example:** NFA for $L_1 L_2 = \{a^n b\} \{ba\} = \{a^n bba\}$

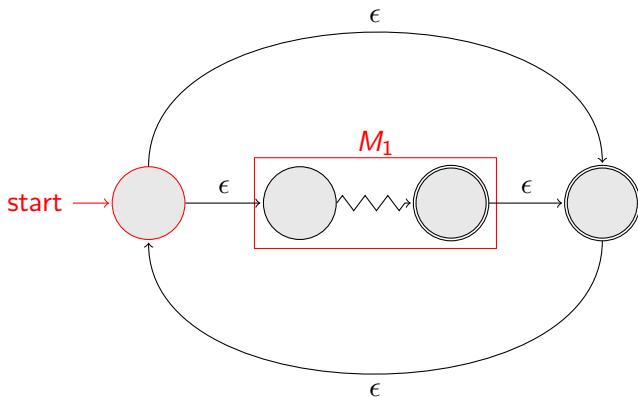


Some Properties of Regular Languages

- **Star Operation:** NFA for L_1^*

Some Properties of Regular Languages

- Star Operation: NFA for L_1^*



Some Properties of Regular Languages

- **Example:** NFA for $L_1^* = \{a^n b\}^*$

Some Properties of Regular Languages

- **Example:** NFA for $L_1^* = \{a^n b\}^*$

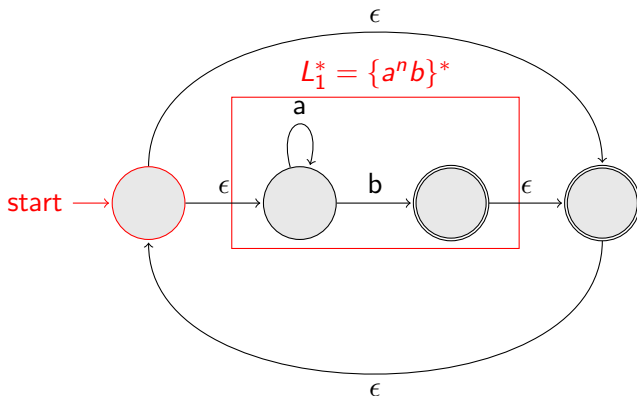


Table of Contents

- 1 Single Final State for NFAs and DFAs
- 2 Some Properties of Regular Languages
- 3 Regular Expressions

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.
- **Regular Expression** is a way to represent a language.

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.
- **Regular Expression** is a way to represent a language.
- **Regular expressions** describe **Regular language**.

Regular Expressions

- The Language accepted by **Finite Automata** are easily described by simple expressions called **Regular Expressions**.
- **Regular Expression** is a way to represent a language.
- **Regular expressions** describe **Regular language**.
- **Example:** $(a + bc)^*$
describes the language
 $\{a, bc\}^* = \{\lambda, a, bc, aa, abc, bca, \dots\}$

Regular Expression

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

$$\left. \begin{array}{l} r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^* \\ (r_1) \end{array} \right\} \text{Are regular expressions}$$

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

$$\left. \begin{array}{l} r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^* \\ (r_1) \end{array} \right\} \text{Are regular expressions}$$

- A regular expression: $(a + bc)^*(c + \Phi)$

Regular Expression

- **Recursive Definition:**

- Primitive regular expressions: Φ, λ, α
- Given Regular Expressions: r_1 & r_2

$$\left. \begin{array}{l} r_1 + r_2 \\ r_1 \cdot r_2 \\ r_1^* \\ (r_1) \end{array} \right\} \text{Are regular expressions}$$

- A regular expression: $(a + bc)^*(c + \Phi)$
- Not a regular expression: $(a+b+)$

Languages of Regular Expressions

- $L(R)$: language of regular expression, r

Example: $L((a + b + c)^*) = \{\lambda, a, bc, aa, abc, bca, \dots\}$

Languages of Regular Expressions

- $L(R)$: language of regular expression, r

Example: $L((a + b + c)^*) = \{\lambda, a, bc, aa, abc, bca, \dots\}$

- For primitive regular expressions:

- $L(\Phi) = \Phi$
- $L(\lambda) = \{\lambda\}$
- $L(a) = \{a\}$
- $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
- $L(r_1.r_2) = L(r_1)L(r_2)$
- $L(r_1^*) = (L(r_1))^*$
- $(L(r_1)) = L(r_1)$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)\}^* = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

- **Regular Expression:** $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &(\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &\{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

- **Regular Expression:** $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

- **Regular Expression:** $r = (0 + 1)^*00(0 + 1)^*$

$$L(r) = \{\text{all strings with at least two consecutive } 0\}$$

Languages of Regular Expressions

- **Example:**

Regular Expression: $(a + b).a^*$

$$\begin{aligned} L((a + b).a^*) &= L(a + b)L(a^*) = (L(a) \cup L(b))L(a^*) = \\ &= (\{a\} \cup \{b\})\{(a)^*\} = \{a, b\}\{\lambda, a, aa, aaa, \dots\} = \\ &= \{a, aa, aaa, \dots, b, bb, bbb, \dots\} \end{aligned}$$

- **Regular Expression:** $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

- **Regular Expression:** $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

- **Regular Expression:** $r = (0 + 1)^*00(0 + 1)^*$

$$L(r) = \{\text{all strings with at least two consecutive } 0\}$$

- **Regular Expression:** $r = (1 + 01)^*(0 + \lambda)$

$$L(r) = \{\text{all strings without two consecutive } 0\}$$

Equivalent Regular Expressions

- Regular expressions r_1 and r_2 are equivalent if $L(r_1) = L(r_2)$
- **Example:** $L = \{ \text{all strings without two consecutive 0} \}$
 $r_1 = (0 + 01)^*(0 + \lambda)$
 $r_2 = (1^*011^*)^*(0 + \lambda) + 1^*(0 + \lambda)$
 $L(r_1) = L(r_2) = L \implies r_1 \text{ and } r_2 \text{ are equivalent regular expression.}$